Table of Contents

- ***Background*** ***6***

1. Data Understanding ***6***

1.1. Information of Column **6**

1.2. Important Columns **7**

1.3. Class Distribution **9**

2. Data Preparation ***10***

2.1. Importing Libraries and Loading the Dataset **10**

2.2. Feature Selection **11**

2.3. Information of Columns after Feature Selection **12**

2.4. Data Cleaning **13**

2.5. Label Renaming **15**

3. Data Analysis ***17***

3.1. Statistics Summary **17**

3.2. Correlation of all numeric values **18**

3.3. *Top 5 correlated features* ***20***

4. Data Exploration ***22***

4.1. Bar chart showing frequency **22**

4.2. Analysis **23**

4.3. Determining the Highest Average Mean **23**

4.4. Showing the Highest Average Mean through a Pie chart 24

4.5. Analysis 26

4.6. Visualizing Distribution through Box Plot 26

4.7. Analysis 28

4.8. Bar chart showing frequency without outliers 29

4.9. Analysis without outliers 30

4.10. Performing a Hypothesis Test 30

- *Appendix A 33*
- *Appendix B 33*
- *Appendix C 33*
- *Appendix D 34*
- *Appendix E 34*
- *Appendix F 35*
- *Appendix G 35*
- *Appendix H 36*
- *Appendix I 36*
- *Appendix J 36*
- *Appendix K 37*
- *Appendix L 37*

Figure 1: Info of Column 6
Figure 2: Importing & Loading dataset 10
Figure 3: Output of the Dataset 11
Figure 4: Selection of Feature 11
Figure 5: Output of Selection Feature 12
Figure 6: Info after Feature Selection 12
Figure 7: Null value Checking 12
Figure 8: Output of null value 13
Figure 9: Removing null values 13
Figure 10: Output After removing null values 14
Figure 11: Checking duplicate records 14
Figure 12: Output of duplicate records 14
Figure 13: Total no of columns 15
Figure 14: Output of columns 15
Figure 15: Name of columns 15
Figure 16: Output of column names 15
Figure 17: Label Distribution 16
Figure 18: Output label distribution 16
Figure 19: Renaming Label 16
Figure 20: Output of renaming label 16
Figure 21: Showing Statistics 17
Figure 22: Output of statistics 18
Figure 23: Correlation of numeric value 18
Figure 24: Output of correlation 19

Figure 25: Heatmap 19
Figure 26: Output of heatmap 20
Figure 27: 5 Correlated feature 20
Figure 28: Output of correlated feature 21
Figure 29: Bar chart showing Frequency 22
Figure 30: Barchart 22
Figure 33: Highest average mean 23
Figure 34: Output of the highest average mean 23
Figure 35: Highest mean through a pie chart 24
Figure 36: Pie chart of average flow duration 24
Figure 37: Packet Length Mean 25
Figure 38: Pie chart of the average packet length mean 25
Figure 39: Visualizing Distribution 27
Figure 40: Boxplot with Outliers 28
Figure 31: Bar chart showing frequency without outliers 29
Figure 32: Barchart without outliers 30
Figure 41: Hypothesis test 31
Figure 42: Output of hypothesis test ..... 32 Table 1: Important Columns 7

• Background

The Jupyter Notebook version 2.3.3 was used as the context to complete this coursework. It is an interactive environment for the Python language used to write and execute code. In this coursework, libraries such as Pandas and NumPy were used to handle and process the data efficiently.

1. Data Understanding

Understanding the data is really important before we start working with it.

A dataset is basically a collection of information that is organized and stored so it can be used later. The dataset we are using for this project has a lot of records about network traffic analysis.AppendixA

1.1. Information of Column

```
df.info()
```

```
Data columns (total 79 columns):
 #   Column                       Non-Null Count   Dtype
---  ------                       --------------   -----
 0   Destination Port             488115 non-null  int64
 1   Flow Duration                488115 non-null  int64
 2   Total Fwd Packets            488115 non-null  int64
 3   Total Backward Packets       488115 non-null  int64
 4   Total Length of Fwd Packets  488115 non-null  int64
 5   Total Length of Bwd Packets  488115 non-null  int64
 6   Fwd Packet Length Max        488115 non-null  int64
 7   Fwd Packet Length Min        488115 non-null  int64
 8   Fwd Packet Length Mean       488115 non-null  float64
 9   Fwd Packet Length Std        488115 non-null  float64
 10  Bwd Packet Length Max        488115 non-null  int64
 11  Bwd Packet Length Min        488115 non-null  int64
 12  Bwd Packet Length Mean       488115 non-null  float64
 13  Bwd Packet Length Std        488115 non-null  float64
 14  Flow Bytes/s                 488079 non-null  float64
 15  Flow Packets/s               488115 non-null  float64
 16  Flow IAT Mean                488115 non-null  float64
 17  Flow IAT Std                 488115 non-null  float64
 18  Flow IAT Max                 488115 non-null  int64
 19  Flow IAT Min                 488115 non-null  int64
 20  Fwd IAT Total                488115 non-null  int64
 21  Fwd IAT Mean                 488115 non-null  float64
 22  Fwd IAT Std                  488115 non-null  float64
 23  Fwd IAT Max                  488115 non-null  int64
 24  Fwd IAT Min                  488115 non-null  int64
 25  Bwd IAT Total                488115 non-null  int64
 26  Bwd IAT Mean                 488115 non-null  float64
 27  Bwd IAT Std                  488115 non-null  float64
 28  Bwd IAT Max                  488115 non-null  int64
 29  Bwd IAT Min                  488115 non-null  int64
 30  Fwd PSH Flags                488115 non-null  int64
 31  Bwd PSH Flags                488115 non-null  int64
 32  Fwd URG Flags                488115 non-null  int64
 33  Bwd URG Flags                488115 non-null  int64
 34  Fwd Header Length            488115 non-null  int64
 35  Bwd Header Length            488115 non-null  int64
 36  Fwd Packets/s                488115 non-null  float64
 37  Bwd Packets/s                488115 non-null  float64
 38  Min Packet Length            488115 non-null  int64
 39  Max Packet Length            488115 non-null  int64
 40  Packet Length Mean           488115 non-null  float64
 41  Packet Length Std            488115 non-null  float64
 42  Packet Length Variance       488115 non-null  float64
 43  FIN Flag Count               488115 non-null  int64
 44  SYN Flag Count               488115 non-null  int64
 45  RST Flag Count               488115 non-null  int64
 46  PSH Flag Count               488115 non-null  int64
 47  ACK Flag Count               488115 non-null  int64
 48  URG Flag Count               488115 non-null  int64
 49  CWE Flag Count               488115 non-null  int64
 50  ECE Flag Count               488115 non-null  int64
 51  Down/Up Ratio                488115 non-null  int64
 52  Average Packet Size          488115 non-null  float64
 53  Avg Fwd Segment Size         488115 non-null  float64
 54  Avg Bwd Segment Size         488115 non-null  float64
 55  Fwd Header Length.1          488115 non-null  int64
 56  Fwd Avg Bytes/Bulk           488115 non-null  int64
 57  Fwd Avg Packets/Bulk         488115 non-null  int64
 58  Fwd Avg Bulk Rate            488115 non-null  int64
 59  Bwd Avg Bytes/Bulk           488115 non-null  int64
 60  Bwd Avg Packets/Bulk         488115 non-null  int64
 61  Bwd Avg Bulk Rate            488115 non-null  int64
 62  Subflow Fwd Packets          488115 non-null  int64
 63  Subflow Fwd Bytes            488115 non-null  int64
 64  Subflow Bwd Packets          488115 non-null  int64
 65  Subflow Bwd Bytes            488115 non-null  int64
 66  Init_Win_bytes_forward       488115 non-null  int64
 67  Init_Win_bytes_backward      488115 non-null  int64
 68  act_data_pkt_fwd             488115 non-null  int64
 69  min_seg_size_forward         488115 non-null  int64
 70  Active Mean                  488115 non-null  float64
 71  Active Std                   488115 non-null  float64
 72  Active Max                   488115 non-null  int64
 73  Active Min                   488115 non-null  int64
 74  Idle Mean                    488115 non-null  float64
 75  Idle Std                     488115 non-null  float64
 76  Idle Max                     488115 non-null  int64
 77  Idle Min                     488115 non-null  int64
 78  Label                        488115 non-null  object
```

Figure 1: Info of Column

1.2. Important Columns

Table 1: Important Columns

Name	Description	Importance
Destination Port	The port number on the destination host where the traffic is going.	Identifies target services; some ports are targets for attacks, like 80, 443 and 22.
Flow Duration	How long the network flow lasts in microseconds.	Long or very short durations can mean attack patterns, like DoS or scanning.
Total Fwd Packets	The total number of packets sent to the direction.	High packet counts in one direction might mean data is being stolen or flooding attacks.
Total Backward Packets	The total number of packets sent to the direction, which is the response.	If there's a difference between packets sent forward and backward it might mean a connection is incomplete or malicious.
Total Length of Fwd Packets	The total size of all packets in bytes.	Large payload volumes might mean data is being transferred or stolen.
Total Length of Bwd Packets	The total size of all packets in bytes.	This helps tell sessions apart from bulk transfers.

Fwd Packet Length Mean	The average size of packets sent to the direction in bytes.	Mean sizes might mean scanning; large values might mean bulk file transfer.
Bwd Packet Length Mean	The average size of packets sent to the direction in bytes.	This shows the server response size; anomalies suggest abnormal behaviour.
Flow Bytes/s	How many bytes are transmitted per second in the flow.	High rates might mean DoS/DDoS attacks or rapid data exfiltration.
Flow Packets/s	How many packets are transmitted per second in the flow.	High rates suggest flooding attacks like SYN flood or UDP flood.
Packet Length Mean	The average packet length across all packets in the flow.	This helps characterize the type of traffic like bulk transfer.
Packet Length Std	The standard deviation of packet lengths.	High variation might mean mixed-content flows or evasion techniques.
Average Packet Size	The average size of all packets in the flow.	This is useful for telling apart application-layer traffic types.
Active Mean	The time the flow was active before becoming idle.	Short active periods might mean port scanning or probe activity.
Flow Bytes/s	How many bytes are transmitted per second in the flow.	Long idle periods can mean low-and-slow attack patterns.

Label	The class label, for each traffic flow: BENIGN or Infiltration.	This is the target variable used for training and evaluating the classification model.

1.3. Class Distribution

The Label column contains two classes:

- BENIGN
- Infiltration AppendixB

2. Data Preparation

2.1. Importing Libraries and Loading the Dataset

• The following code loads the dataset and displays the last 50 records using the tail() method.

```
import pandas as pd
import scipy.stats as stats
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import ttest_ind
```

Loading and displaying data

```
df = pd.read_csv('ids.csv', index_col=0) #Load Dataset
```

```
df.tail(50) # Display last 50 records
```

Figure 2: Importing & Loading dataset

Explanation of importing & Loading dataset

- Pd.read_csv reads the CSV file and loads the dataset into a pandas DataFrame.
- Df.tail(50) returns the last 50 records and displays.

Output

	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Max	Fwd Packet Length Min	Fwd Packet Length Mean	Fwd Packet Length Std	...	min_seg_size_forward	Active Mean	A
488065	53	70591	2	2	66	194	33	33	33.000000	0.000000	...	32	0.00000	
488066	443	5572560	8	6	386	5205	205	0	48.250000	75.952523	...	20	0.00000	
488067	3013	49	2	0	4	0	2	2	2.000000	0.000000	...	24	0.00000	
488068	53	122130	2	2	80	272	40	40	40.000000	0.000000	...	20	0.00000	
488069	53	98106274	2	2	114	211	58	56	57.000000	1.414214	...	20	30943.00000	
488070	53	506	1	1	53	113	53	53	53.000000	0.000000	...	32	0.00000	
488071	53	3957631	2	2	120	260	79	41	60.000000	26.870058	...	20	0.00000	
488072	9102	2	2	0	4	0	2	2	2.000000	0.000000	...	24	0.00000	
488073	2135	111	2	2	4	12	2	2	2.000000	0.000000	...	24	0.00000	
488074	1048	49	2	0	4	0	2	2	2.000000	0.000000	...	24	0.00000	
488075	54240	55	1	1	6	6	6	6	6.000000	0.000000	...	20	0.00000	
488076	1049	73	2	2	4	12	2	2	2.000000	0.000000	...	24	0.00000	
488077	443	4	2	0	12	0	6	6	6.000000	0.000000	...	20	0.00000	
488078	53	172	2	2	80	112	40	40	40.000000	0.000000	...	20	0.00000	
488079	443	79063	1	2	0	0	0	0	0.000000	0.000000	...	32	0.00000	

Figure 3: Output of the Dataset

2.2. Feature Selection

- The following code makes a new data frame containing only 16 specified columns.

```
columns = ['Destination Port', 'Flow Duration', 'Total Fwd Packets',
           'Total Backward Packets', 'Total Length of Fwd Packets',
           'Total Length of Bwd Packets', 'Fwd Packet Length Mean',
           'Bwd Packet Length Mean', 'Flow Bytes/s', 'Flow Packets/s',
           'Packet Length Mean', 'Packet Length Std', 'Average Packet Size',
           'Active Mean', 'Idle Mean', 'Label'] # List of required Columns
new_df = df[columns] # Create a new Dataframe with Selected Columns
new_df.head() # Display First 5 records
```

Figure 4: Selection of Feature

Explanation of Feature Selection

- The Python list columns contained only 16 specified columns, Appendix C.

Output

	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Mean	Bwd Packet Length Mean	Flow Bytes/s	Flow Packets/s	Packet Length Mean	Packet Length Std	Average Packet Size
0	53	62171	2	2	78	164	39.00	82.000000	3.892490e+03	64.338679	56.200000	23.552070	70.250000
1	2710	48	2	0	4	0	2.00	0.000000	8.333333e+04	41666.666670	2.000000	0.000000	3.000000
2	443	4792909	5	1	135	46	27.00	46.000000	3.776412e+01	1.251849	32.428571	18.866700	37.833333
3	80	115596470	75	82	342	118155	4.56	1440.914634	1.025092e+03	1.358173	749.981013	946.124811	754.757962
4	2910	14	2	2	4	12	2.00	6.000000	1.142857e+06	285714.285700	3.600000	2.190890	4.500000

Figure 5: Output of Selection Feature

2.3. Information of Columns after Feature Selection

```
new_df.info()
```

```
Data columns (total 16 columns):
 #   Column                       Non-Null Count   Dtype
---  ------                       --------------   -----
 0   Destination Port             488079 non-null  int64
 1   Flow Duration                488079 non-null  int64
 2   Total Fwd Packets            488079 non-null  int64
 3   Total Backward Packets       488079 non-null  int64
 4   Total Length of Fwd Packets  488079 non-null  int64
 5   Total Length of Bwd Packets  488079 non-null  int64
 6   Fwd Packet Length Mean       488079 non-null  float64
 7   Bwd Packet Length Mean       488079 non-null  float64
 8   Flow Bytes/s                 488079 non-null  float64
 9   Flow Packets/s               488079 non-null  float64
 10  Packet Length Mean           488079 non-null  float64
 11  Packet Length Std            488079 non-null  float64
 12  Average Packet Size          488079 non-null  float64
 13  Active Mean                  488079 non-null  float64
 14  Idle Mean                    488079 non-null  float64
 15  Label                        488079 non-null  object
```

Figure 6: Info after Feature Selection

- The following code checks for null values in the data set.

```
new_df.isna().sum() # Checking for Null Values
```

Figure 7: Null value Checking

Explanation of null value checking

- Isna() checks for the null values presented in the dataset.
- Sum() gives the total count of null values presented in the code.

Output

```
Destination Port               0
Flow Duration                  0
Total Fwd Packets              0
Total Backward Packets         0
Total Length of Fwd Packets    0
Total Length of Bwd Packets    0
Fwd Packet Length Mean         0
Bwd Packet Length Mean         0
Flow Bytes/s                  36
Flow Packets/s                 0
Packet Length Mean             0
Packet Length Std              0
Average Packet Size            0
Active Mean                    0
Idle Mean                      0
Label                          0
dtype: int64
```

Figure 8: Output of null value

2.4. Data Cleaning

- The following code deletes rows containing null values and rechecks the same code used to verify that the null values have been deleted.

```
new_df = new_df.dropna() # Remove rows with Null Values
```

```
new_df.isna().sum() # Rechecking after removing
```

Figure 9: Removing null values

Output

```
Destination Port              0
Flow Duration                 0
Total Fwd Packets             0
Total Backward Packets        0
Total Length of Fwd Packets   0
Total Length of Bwd Packets   0
Fwd Packet Length Mean        0
Bwd Packet Length Mean        0
Flow Bytes/s                  0
Flow Packets/s                0
Packet Length Mean            0
Packet Length Std             0
Average Packet Size           0
Active Mean                   0
Idle Mean                     0
Label                         0
dtype: int64
```

Figure 10: Output After removing null values

- The following code checks if there is duplicate data presented in the dataset.

```
new_df.duplicated().sum() # Check if Duplicate records exist
```

Figure 11: Checking duplicate records

Explanation of checking duplicate records

- Duplicated() checks for duplicate data present in the dataset.
- Sum() gives the total count of the duplicated data that is in the dataset.

Output

```
np.int64(139389)
```

Figure 12: Output of duplicate records

• The following code gives the total number of columns presented in the dataset.

```
len(new_df.columns) # Display total number of Columns
```

Figure 13: Total no of columns

Output

```
16
```

Figure 14: Output of columns

• The following code gives the list of column names of the new dataframe.

```
new_df.columns # Display name of all Columns
```

Figure 15: Name of columns

Output

```
Index(['Destination Port', 'Flow Duration', 'Total Fwd Packets',
       'Total Backward Packets', 'Total Length of Fwd Packets',
       'Total Length of Bwd Packets', 'Fwd Packet Length Mean',
       'Bwd Packet Length Mean', 'Flow Bytes/s', 'Flow Packets/s',
       'Packet Length Mean', 'Packet Length Std', 'Average Packet Size',
       'Active Mean', 'Idle Mean', 'Label'],
      dtype='object')
```

Figure 16: Output of column names

2.5. Label Renaming

• The following code counts the value presented in the label column.

Explanation of label value counts

- New_df['Label'] gives access to the label column.
- Value_counts() gives the total count of the values presented in the label column.

```
new_df['Label'].value_counts() # Display original Label Distribution
```

Figure 17: Label Distribution

Output

```
Label
BENIGN    288548
Attack    199531
Name: count, dtype: int64
```

Figure 18: Output label distribution

- The following code renames the label values: ‘Benign’ to ‘Normal Traffic,’ and ‘Infiltration’ to ‘Attack.

```
new_df['Label'] = new_df['Label'].replace({ # Rename Label Values
    'Benign': 'Normal Traffic',
    'Infiltration': 'Attack'
})
new_df['Label'].value_counts() # Display updated Label Distribution
```

Figure 19: Renaming Label

Output

```
Label
Normal Traffic    288548
Attack            199531
Name: count, dtype: int64
```

Figure 20: Output of renaming label

3. Data Analysis

3.1. Statistics Summary

- The code below performs a statistical analysis of 5 essential metrics for packet transmission from source to destination in both directions using a "for loop".

Explanation of statistics

- The dataframe being analyzed can be found in the prepared dataframe, which has multiple columns (Total Packets Sent and Total Packets Received).
- For each column, 5 statistical metrics are calculated and displayed in Appendix D.

```
columns1 = ['Total Fwd Packets', 'Total Backward Packets'] # Columns to analyze
```

```
for col in columns1: # Loop to calculate basic statistics
    print("\n", col, "")
    print("Sum:", df[col].sum())
    print("Mean:", df[col].mean())
    print("Standard Deviation:", df[col].std())
    print("Skewness:", stats.skew(df[col]))
    print("Kurtosis:", stats.kurtosis(df[col]))
```

Figure 21: Showing Statistics

Output

- Both sets of data (Total Packets Sent and Received) exhibited a very high amount of right-skewed data and exhibited a very high amount of kurtosis when the distributions were analyzed.

```
 Total Fwd Packets
Sum: 3029696
Mean: 6.206930743779642
Standard Deviation: 69.85269062816748
Skewness: 137.04269372657225
Kurtosis: 30408.018375304313

 Total Backward Packets
Sum: 3019274
Mean: 6.1855792180121485
Standard Deviation: 99.35912082347987
Skewness: 227.4740316100872
Kurtosis: 87218.69394568843
```

Figure 22: Output of statistics

3.2. Correlation of all numeric values

• According to the Pearson correlation matrix code:

Explanation of correlation

- The DataFrame is filtered to include only numeric columns and exclude categorical variables (e.g. Label) using "select_dtypes(include='number')".
- The Pearson correlation coefficient (ranging from -1 to +1) between each pair of numeric features is computed using "new_df1.corr()".

```
new_df1 = new_df.select_dtypes(include = 'number') # Selects only numeric columns
new_df1.corr()
```

Figure 23: Correlation of numeric value

Output

	Destination Port	Flow Duration	Total Fwd Packets	Total Backward Packets	Total Length of Fwd Packets	Total Length of Bwd Packets	Fwd Packet Length Mean	Bwd Packet Length Mean	Flow Bytes/s	Flow Packets/s	Packet Length Mean	Packet Length Std	Average Packet Size
Destination Port	1.000000	-0.145250	-0.025711	-0.024028	0.004815	-0.015581	0.005199	-0.217264	0.063783	0.214177	-0.118523	-0.156795	-0.127719
Flow Duration	-0.145250	1.000000	0.124344	0.100836	0.032044	0.063465	0.114950	0.351037	-0.018884	-0.113359	0.324827	0.465984	0.303080
Total Fwd Packets	-0.025711	0.124344	1.000000	0.949848	0.333574	0.920007	0.035838	0.210119	-0.002078	-0.020480	0.200438	0.174919	0.190185
Total Backward Packets	-0.024028	0.100836	0.949848	1.000000	0.146728	0.961480	0.015679	0.200844	-0.002868	-0.020564	0.186629	0.156695	0.177268
Total Length of Fwd Packets	0.004815	0.032044	0.333574	0.146728	1.000000	0.013944	0.127556	0.016333	0.001838	-0.006509	0.080155	0.054492	0.077949
Total Length of Bwd Packets	-0.015581	0.063465	0.920007	0.961480	0.013944	1.000000	-0.001047	0.194512	-0.001385	-0.012061	0.176026	0.138066	0.167419
Fwd Packet Length Mean	0.005199	0.114950	0.035838	0.015679	0.127556	-0.001047	1.000000	0.123866	0.154969	-0.078665	0.612858	0.462664	0.628276
Bwd Packet Length Mean	-0.217264	0.351037	0.210119	0.200844	0.016333	0.194512	0.123866	1.000000	-0.024280	-0.166846	0.824952	0.876817	0.809420
Flow Bytes/s	0.063783	-0.018884	-0.002078	-0.002868	0.001838	-0.001385	0.154969	-0.024280	1.000000	0.200319	0.127875	0.069086	0.188463
Flow Packets/s	0.214177	-0.113359	-0.020480	-0.020564	-0.006509	-0.012061	-0.078665	-0.166846	0.200319	1.000000	-0.134655	-0.145674	-0.135588
Packet Length Mean	-0.118523	0.324827	0.200438	0.186629	0.080155	0.176026	0.612858	0.824952	0.127875	-0.134655	1.000000	0.887792	0.995068
Packet													

Figure 24: Output of correlation

Explanation of heatmap

AppendixE

```
plt.figure(figsize=(12,8))
sns.heatmap(new_df1.corr(), annot = True) #Show correlation values
```

Figure 25: Heatmap

Output

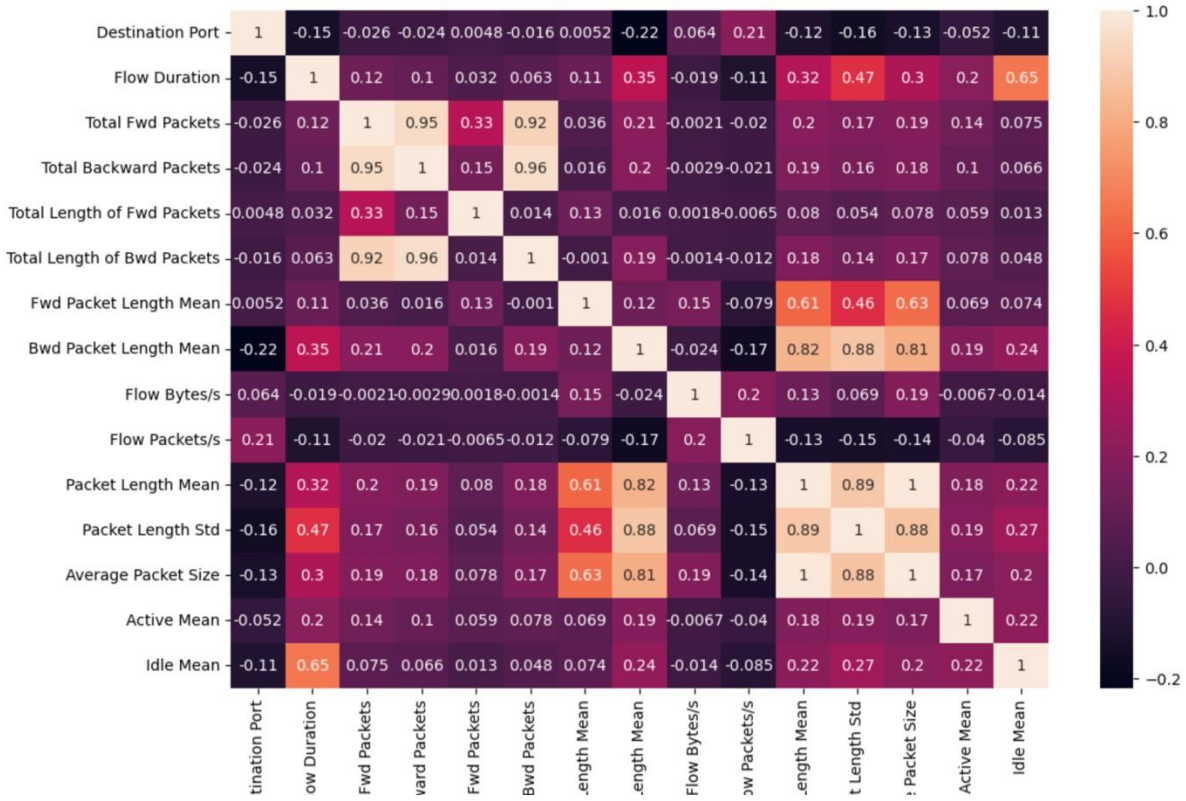


Figure 26: Output of heatmap

3.3. Top 5 correlated features

• The resulting code converts the correlation matrix into ranked feature pairs and captures five pairs of features with the highest correlation values.

Explanation of the 5 correlated features

AppendixF

```
new_df1 = new_df.select_dtypes(include='number')

corr_matrix = new_df1.corr() # Find correlation matrix

corr_pairs = corr_matrix.unstack() #Converts matrix to pairs

corr_pairs = corr_pairs[corr_pairs != 1] # Remove same variable correlation(value = 1)

corr_pairs = corr_pairs.drop_duplicates() # Remove duplicate pairs

top_5 = corr_pairs.reindex(corr_pairs.abs().sort_values(ascending=False).index) # Sort to find highest correlation

print("Top 5 Correlated Feature Pairs:\n") # Print top 5 correlated feature
print(top_5.head(5))
```

Figure 27: 5 Correlated feature

Output

```
Top 5 Correlated Feature Pairs:

Packet Length Mean      Average Packet Size           0.995068
Total Backward Packets  Total Length of Bwd Packets   0.961480
Total Fwd Packets       Total Backward Packets        0.949848
                        Total Length of Bwd Packets   0.920007
Packet Length Mean      Packet Length Std             0.887792
dtype: float64
```

Figure 28: Output of correlated feature

4. Data Exploration

4.1. Bar chart showing frequency

• The code shown below counts the occurrences of each label category and produces a bar chart of the counts.

Explanation

AppendixG

```
label_counts = new_df['Label'].value_counts() # Count number of each label

plt.figure(figsize=(6,5))
plt.bar(label_counts.index, label_counts.values) # Bar chart for level frequency

plt.title("Frequency of Label Categories")
plt.xlabel("Label")
plt.ylabel("Count")

plt.show()
```

Figure 29: Bar chart showing Frequency

Output

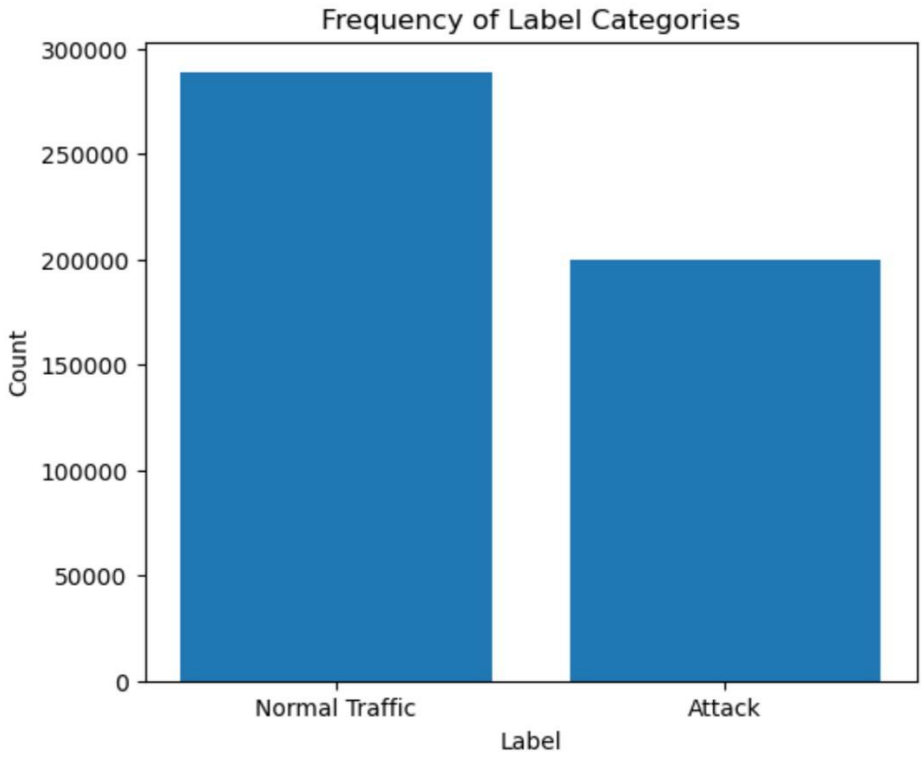


Figure 30: Barchart

4.2. Analysis

- In the above Bar Graph, approximately 290,000 Normal Traffic records and 200,000 Attack records were identified through data exploration.
- This represents a 59 percent ratio of Normal Traffic to Attack records and thus indicates that there is a Moderate Imbalance present within the dataset.

4.3. Determining the Highest Average Mean

• The DataFrame is split into two groups based upon the value within the Label column (i.e., Normal Traffic and Attack), based upon Flow Duration and Packet Length Mean within each group.

Explanation of highest average mean

- The function groupby('Label') creates two distinct DataFrames: one containing all Normal Traffic and one containing all Attack records.
- The resulting DataFrames created through this grouping have all records for each group that correspond with those values, and with respect to Flow Duration and Packet Length Mean.

AppendixH

```
group = new_df.groupby('Label')[['Flow Duration', 'Packet Length Mean']].mean() # Group data by label and find average values
group
```

Figure 31: Highest average mean

Output

Label	Flow Duration	Packet Length Mean
Attack	8.909046e+06	85.573255
Normal Traffic	8.966348e+06	85.542480

Figure 32: Output of the highest average mean

4.4. Showing the Highest Average Mean through a Pie chart

Explanation of the flow duration through a pie chart

- Using the plt.pie() method creates a pie chart that represents the average Flow Duration values based on the various labels present in the dataset. Appendixl

```
plt.figure()
plt.pie(group['Flow Duration'], labels = group.index, autopct = '%1.1f%%') # Pie chart for flow duration
plt.title("Average Flow Duration by Label")
plt.show()
```

Figure 33: Highest mean through a pie chart

Output

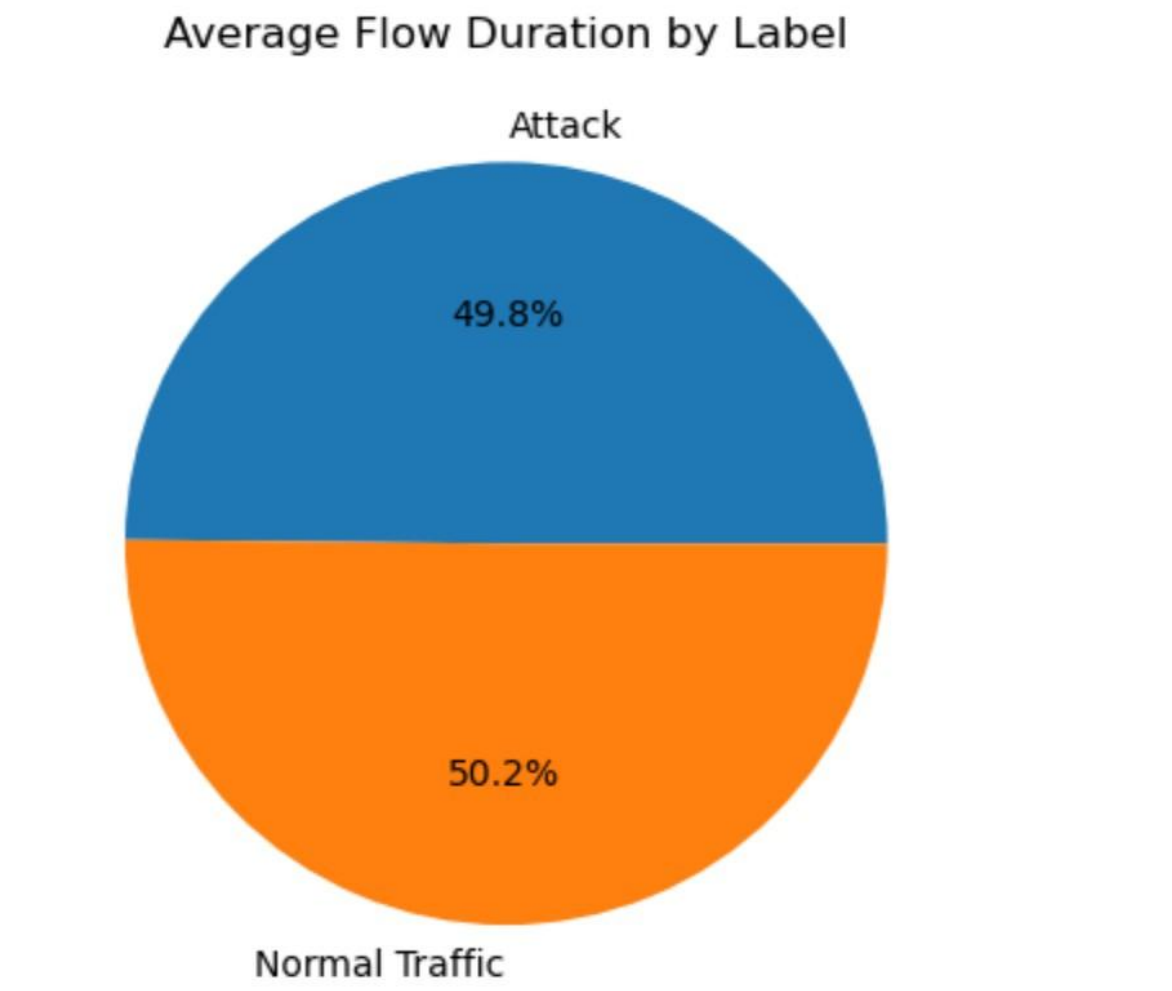


Figure 34: Pie chart of average flow duration

Explanation of Packet Length Mean

- The same use of the pie chart format is used to visually represent the proportional distribution of the average length of the packets for Attack Traffic versus Normal Traffic.

```
plt.figure()
plt.pie(group['Packet Length Mean'], labels = group.index, autopct= '%1.1f%%') # Pie chart for packet length mean
plt.title("Average Packet Length Mean by Label")
plt.show()
```

Figure 35: Packet Length Mean

Output

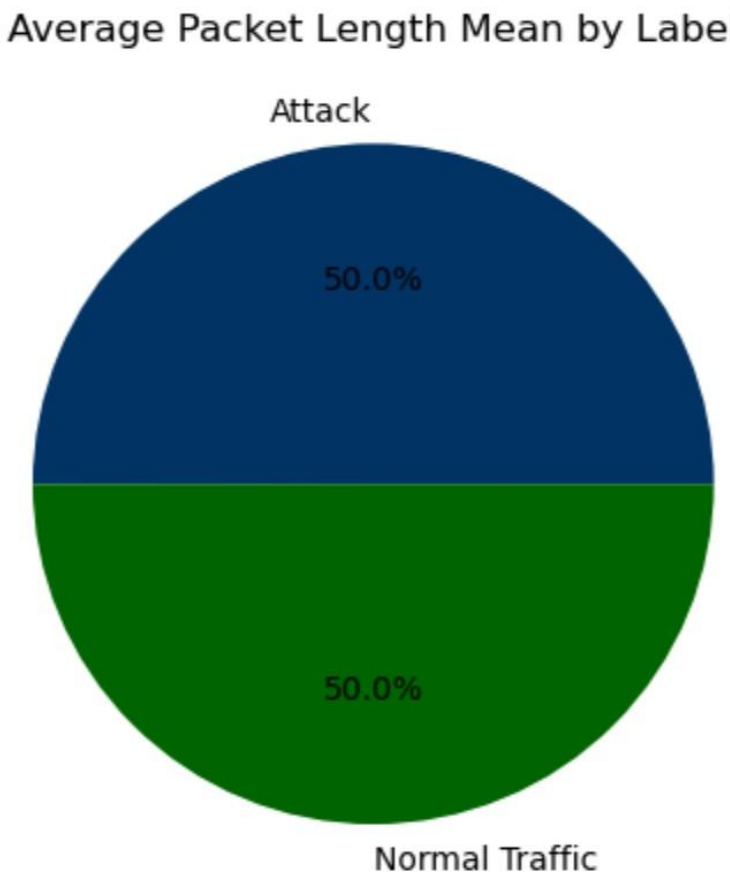


Figure 36: Pie chart of the average packet length mean

4.5. Analysis

- The proportion of the average flow durations for the two labels/traffic types

(Attack and Normal) was approximately equal. The average Flow Duration for Normal traffic was 50.2%, while for Attack was 49.8%. Therefore, the difference in average flow durations was approximately 8.97 million vs 8.91 million microseconds. This means that for this particular feature, the two classes are very similar in terms of their average flow durations.

- The proportional length of packets on average from both attack and normal traffic types is also quite similar. The average Packet Length from both groups was also equivalent, with a Packet Length Mean of approximately 50.0 % and 50.0 %, averaging 85.54 bytes and 85.57 bytes. Therefore, there is little to no difference between the two classes on average when these features are compared. Hence, the average Flow Duration and average Packet Length alone are weak indicators of discriminating normal traffic from attack traffic.

4.6. Visualizing Distribution through Box Plot

- To display the differences in distribution for the Forward Packet Length Mean feature within each traffic type, A box plot is created for that comparative feature.

Explanation of box plot distribution

- The sns. boxplot() command from the seaborn library produces a grouped boxplot via an array of data by utilizing the parameter x='Label' to create different categories from the x-axis.

AppendixJ

```
# Box plot to compare data distribution
plt.figure(figsize=(8,6))
sns.boxplot(
    x='Label',
    y='Fwd Packet Length Mean',
    data= new_df
)
plt.title("Fwd Packet Length Mean Grouped by Label")
plt.xlabel("Label")
plt.ylabel("Fwd Packet Length Mean")
plt.show()
```

Figure 37: Visualizing Distribution

Output

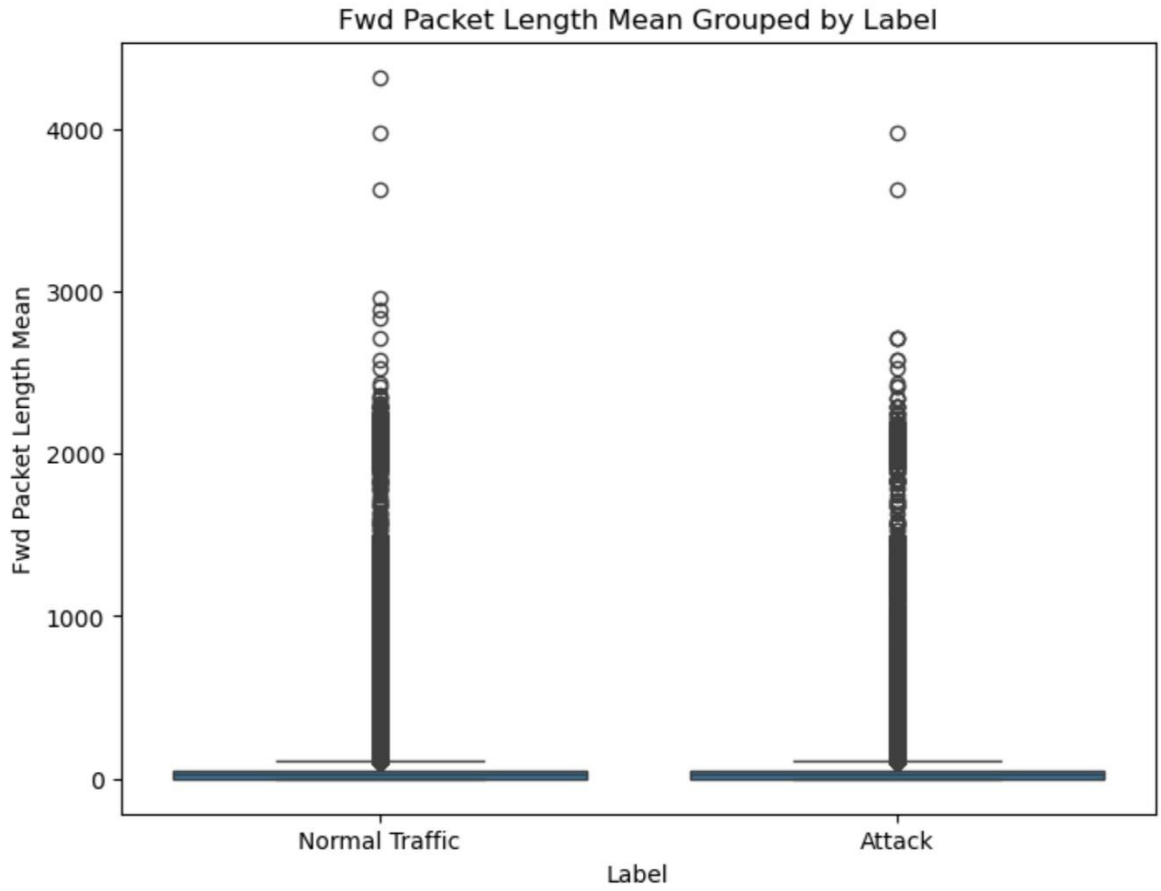


Figure 38: Boxplot with Outliers

4.7. Analysis

- Both the Normal Traffic and Attack categories exhibit a very small compressed interquartile range, which will be shown near 0. This indicates there is a majority of flows that have very small forward packet lengths, regardless of category.
- The distribution for both Normal Traffic and Attack categories shows significant right skew, as indicated by the presence of many outlier points that extend to approximately 4,000–4,300 bytes. This indicates that there are very few flows that have very large forward packets for both categories.
- Both the median values of the two Groups were very similar and slightly above 0 with respect to their outliers. Hence, we can conclude that Fwd Packet Length Mean cannot be used alone to identify attack traffic, as compared to normal traffic.

- The skewness (137.04) and kurtosis (30,408) statistics indicate that this data set has extreme levels of non-normality.

4.8. Bar chart showing frequency without outliers

- The code shown below counts the occurrences of each label category and produces a bar chart of the counts without outliers.

```
# Box plot without outliers
plt.figure(figsize=(8,6))

sns.boxplot(
    x='Label',
    y='Fwd Packet Length Mean',
    data=new_df,
    showfliers=False
)

plt.title("Fwd Packet Length Mean Grouped by Label")
plt.xlabel("Label")
plt.ylabel("Fwd Packet Length Mean")

plt.show()
```

Figure 39: Bar chart showing frequency without outliers

Output

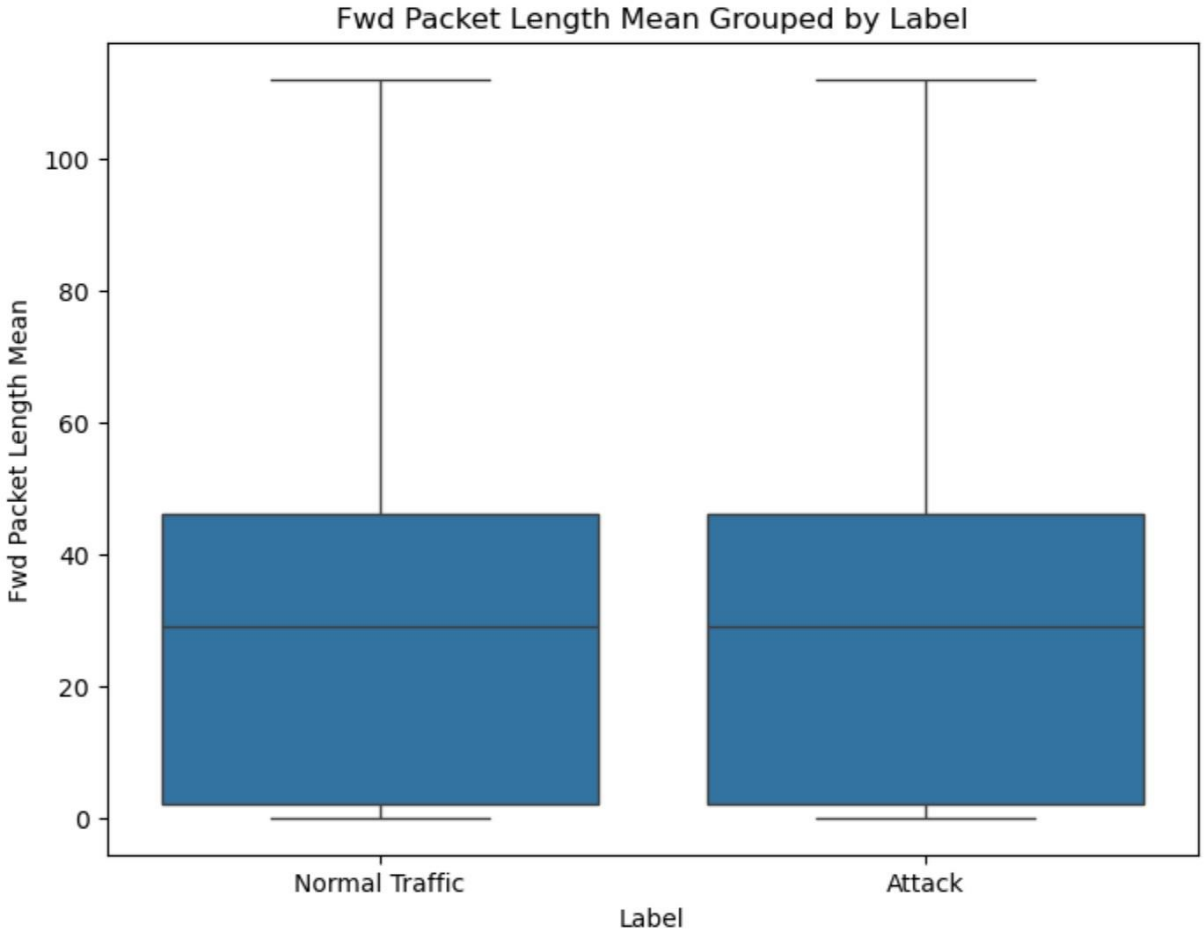


Figure 40: Barchart without outliers

4.9. Analysis without outliers

- When we look at the distributions of normal and attack traffic, they have practically identical interquartile ranges and the same whisker lengths.
- The difference between the distributions of normal and attack traffic is negligible.

4.10. Performing a Hypothesis Test

Hypothesis

- The Null Hypothesis (H_0) states that there will be no significant difference between the two sets of data; Normal Traffic and Attack will produce the same Flow Duration average.

- The Alternative Hypothesis (H_1) states that there are significant differences between the two groups.

Test Selection & Code

- The independent sample t-test is the proper test for use since both sets are created from independent groups, and we are comparing the means of two different continuous variables.
- The t-test was specifically designed for comparing the mean flow duration from normal traffic with the mean flow duration associated with an attack on that traffic.
- All flows belong to either one of these independent groups (normal OR attack), and no flow can belong to more than one of these two groups.
- This meets the requirement for the independent assumption of the t-test.

Explanation of the hypothesis test

AppendixK

```
# Separate normal and attack data
normal = new_df[new_df['Label'] == 'Normal Traffic']['Flow Duration']
attack = new_df[new_df['Label'] == 'Attack']['Flow Duration']

# Calculate mean
normal_mean = normal.mean()
attack_mean = attack.mean()

t_stat, p_val = ttest_ind(normal,attack) # Perform ttest

# Print result
print("Mean (Normal Traffic):", normal_mean)
print("Mean (Attack):", attack_mean)
print("P-value is:", p_val)

#Decision
if p_val < 0.05:
    print("Reject the null hypothesis")

else:
    print("Fail to reject the null hypothesis")
```

Figure 41: Hypothesis test

Results

- Normal Traffic Average = 8,966,348.48 microseconds.
- Attack Traffic Average = 8,909,046.44 microseconds.
- P-value = 0.4739.
- Decision = Fail to reject the null hypothesis.

Interpretation

AppendixL

```
Mean (Normal Traffic): 8966348.484681925
Mean (Attack): 8909046.444346994
P-value is: 0.473909720385436
Fail to reject the null hypothesis
```

Figure 42: Output of hypothesis test

- **Appendix A**

Data Understanding

We need to take a look at the data to see what it is made of, what kind of information it has, and what the important parts are. This means checking the structure of the data, the types of information it has, and the key features, like the columns. We also need to find any patterns, missing information, or mistakes.

It is a popular dataset that people use to test and build systems that can detect intrusions, which is a big part of network security and machine learning. The dataset has a lot of information about network traffic, including traffic and traffic that is trying to infiltrate the system. Network traffic analysis is a part of this dataset, and it is used to build and evaluate systems that can detect intrusions like intrusion detection systems.

- **Appendix B**

Class Distribution

- BENIGN – Represents normal, legitimate network traffic (renamed to 'Normal Traffic' in preprocessing).
- Infiltration – Represents malicious or attack traffic (renamed to 'Attack' in preprocessing).

- **Appendix C**

Explanation of Feature Selection

- Df[columns] makes a new data frame containing only specified columns from the original.
- New_df.head() displays the first 5 records to check if the columns are made.

• Appendix

Explanation

•

D

of Statistics Summary

For each column, the sum function is used to count the total number of packets sent in all packets sent or packets received when all packets are combined.

- The mean function is found in Python's pandas module and is used to find the mean packet transmitted.
- The std() function found in pandas is used to calculate the standard deviation of the packets sent from their mean.
- Skew() function from scipy.stats package is used to calculate how "skewed" the dataset is or how asymmetrical the distribution is with respect to the average. The greater the value of skew, the more the distribution of an observed value differs from its true mean.
- Kurtosis() from scipy.stats calculates how "tailed" the data is and how "peaked" it is (or affected by extreme values).

• Appendix E

Explanation of Correlation

- A Seaborn heatmap is a visual representation of the entire correlation matrix.
- The dimensions of the graph are set to be 12 inches wide and 8 inches tall to ensure that all feature labels are legible.
- Sns.heatmap() displays the correlation between each feature as an image with colour coding; colour-coded images depict how positively correlated a feature is with another. Warmer colours (red/yellow-ish) represent strongly positively correlated features, while cooler colours (blue, etc.) will represent features that are negatively correlated (to each other).

• Appendix

Explanation

•

- With 'annot=True', we can see the actual value of Pearson correlation coefficients in each of the cells in the heatmap.

F

of the Top 5 Correlated Features

The method "corr_matrix.unstack()" changes the matrix from a two-dimensional array to a one-dimensional Series of (feature 1):(feature 2) correlation pairs.

- The method "corr_pairs != 1" filters out self-correlations, where any feature is perfectly correlated to itself (r = 1.0).
- The method "drop_duplicates()" removes 'mirror duplicates.' Because of the symmetry of correlation (A↔B = B↔A), each pair appears twice in the unstacked list of correlation pairs.
- The method "abs().sort_values(ascending=False)" sorts the correlation pairs by absolute value in order, with strong (positive or negative) correlations appearing first.
- The method "top_5.head(5)" prints only the top five pairs of correlated features.

Output of the Top 5 Correlated Features

- Packet Length Mean & Average Packet Size = r = 0.9951
- Total Backward Packets & Total Length of Bwd Packets = r = 0.9615
- Total Fwd Packets & Total Backward Packets = r = 0.9498
- Total Fwd Packets & Total Length of Bwd Packets = r = 0.9200
- Packet Length Mean & Packet Length Std = r = 0.8878

- **Appendix**

Explanation

-

- **Appendix G**

Explanation of the bar chart showing Frequency

- The pandas function value_counts() returns a Series that holds the number of times each unique value appears within a given DataFrame, Label column.
- The matplotlib function plt.bar() uses each label name as an x-axis category and their respective value counts as the bar height.
- The matplotlib functions plt.title(), plt.xlabel(), and plt.ylabel() help to label the bars on the chart.

H

of determining the highest average mean

By specifying [['Flow Duration', 'Packet Length Mean']], you will select only these two columns from the DataFrames generated through groupby().

- The .mean() function will then return the average of each specified column for each group (i.e., two averages per column).

- **Appendix I**

Explanation of the highest average through a pie chart

- The Attack and Normal Traffic slices are identified using the labels=group.index command.
- For each slice, the autopct='%1.1f%%' portion of the method formats the percentage to one decimal point and displays it on the slice itself.

- **Appendix**

Explanation

-

- **Appendix J**

Explanation of the box plot through the distribution

- The command also designates a continuous variable for the vertical axis by utilizing the parameter y='Fwd Packet Length Mean'.
- The IQR range is defined as the range between the 25th and 75th percentiles. For each category, a box is represented.
- The median is represented by the horizontal line inside the box.
- Whiskers will extend from each box to a maximum length of 1.5 x IQR.
- Any data point that lies beyond the whisker’s extension will be classified as an outlier.

Appendix K

Explanation of the test selection & code

- The data collected were classed into two independent series based on the Label column (i.e., one series will consist of normal traffic data and the other will consist of attack data). The mean for each class was calculated using the mean method within the data frame for each traffic class.
- By using the ttest_ind function, we used both sets previously defined to calculate the t-distribution statistic and the corresponding t-distribution p-value.
- An if/else block is created to compare the p-value against the 0.05 significance threshold so that as soon as it is executed, the conclusion (i.e., fail to reject the Null Hypothesis) can be printed.

Appendix L

Interpretation of Hypothesis Test

- As the p-value of 0.4739 is much larger than the alpha ($\alpha = 0.05$), we do not have enough evidence to reject the null hypothesis.
- Although the difference in class means is 57,302 microseconds approximately, this difference is not statistically significant; thus, it could happen by chance.
- Flow Duration by itself does not allow us to distinguish between the populations of Attack and Normal Traffic in a reliable way.